

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">

  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <meta name="description"
    content="Interactive JavaScript To-Do List with CRUD, localStorage,
filtering and DOM manipulation">

  <title>Smart Task Manager</title>

  <style>

    /* =====
      CSS VARIABLES
    ===== */

    :root {
      --primary: #2563eb;
      --primary-dark: #1d4ed8;
      --success: #16a34a;
      --danger: #dc2626;
      --warning: #f59e0b;

      --background: #f1f5f9;
      --card: #ffffff;
      --text: #172033;
      --muted: #64748b;
      --border: #dbe3ef;

      --shadow:
        0 10px 30px rgba(0, 0, 0, 0.08);

      --radius: 14px;
    }

    /* =====
      RESET
    ===== */

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
  
```

```

body {

    font-family:
        Arial,
        Helvetica,
        sans-serif;

    background:
        linear-gradient(
            135deg,
            #eff6ff,
            #f8fafc
        );

    color: var(--text);

    min-height: 100vh;

    padding: 30px 15px;
}

button,
input {
    font: inherit;
}

button {
    cursor: pointer;
}

/* =====
   MAIN CONTAINER
   ===== */

.app {

    width: 100%;

    max-width: 900px;

    margin: auto;

    background: var(--card);

    border-radius: 20px;

```

```

        box-shadow: var(--shadow);

        overflow: hidden;
    }

    /* =====
    HEADER
    ===== */

    .header {

        background:
            linear-gradient(
                135deg,
                #2563eb,
                #4f46e5
            );

        color: white;

        padding: 35px 30px;

        text-align: center;
    }

    .header h1 {

        font-size: 36px;

        margin-bottom: 8px;
    }

    .header p {

        opacity: 0.9;

        font-size: 16px;
    }

    /* =====
    CONTENT
    ===== */

    .content {

```

```

        padding: 30px;
    }

    /* =====
      ADD TASK
    ===== */

    .add-task {
        display: flex;
        gap: 10px;
        margin-bottom: 25px;
    }

    .add-task input {
        flex: 1;
        padding: 14px 16px;
        border:
            2px solid var(--border);
        border-radius: 10px;
        outline: none;
        color: var(--text);
        background: white;
    }

    .add-task input:focus {
        border-color:
            var(--primary);
    }

    .add-button {
        background:
            var(--primary);
        color: white;
    }

```

```

    border: none;

    padding:
      0 25px;

    border-radius: 10px;

    font-weight: bold;
  }

.add-button:hover {

  background:
    var(--primary-dark);
}

/* =====
  SEARCH
===== */

.search-box {

  margin-bottom: 20px;
}

.search-box input {

  width: 100%;

  padding: 13px 15px;

  border:
    2px solid var(--border);

  border-radius: 10px;

  outline: none;
}

.search-box input:focus {

  border-color:
    var(--primary);
}

```

```
/* =====  
    FILTER BUTTONS  
===== */  
  
.filters {  
    display: flex;  
    gap: 10px;  
    flex-wrap: wrap;  
    margin-bottom: 25px;  
}  
  
.filter-button {  
    border:  
        1px solid var(--border);  
  
    background: white;  
  
    color: var(--text);  
  
    padding:  
        9px 18px;  
  
    border-radius: 20px;  
  
    font-weight: bold;  
}  
  
.filter-button:hover {  
    border-color:  
        var(--primary);  
  
    color:  
        var(--primary);  
}  
  
.filter-button.active {  
    background:  
        var(--primary);
```

```

        color: white;

        border-color:
            var(--primary);
    }

    /* =====
    TASK LIST
    ===== */

    .task-list {

        list-style: none;

        display: flex;

        flex-direction: column;

        gap: 12px;
    }

    /* Dynamic DOM Task */

    .task-item {

        display: flex;

        align-items: center;

        gap: 12px;

        padding: 16px;

        border:
            1px solid var(--border);

        border-radius: 12px;

        background: #ffffff;

        transition:
            transform 0.2s ease,
            box-shadow 0.2s ease;
    }

    .task-item:hover {

```

```

        transform:
            translateY(-2px);

        box-shadow:
            0 5px 15px
            rgba(0,0,0,0.07);
    }

/* =====
CHECKBOX
===== */

.task-check {

    width: 20px;

    height: 20px;

    cursor: pointer;

    accent-color:
        var(--success);

    flex-shrink: 0;
}

/* =====
TASK TEXT
===== */

.task-text {

    flex: 1;

    word-break: break-word;

    font-size: 16px;
}

.task-item.completed
.task-text {

    text-decoration:
        line-through;

    color:
        var(--muted);

```

```
}

/* =====
   TASK ACTIONS
   ===== */

.task-actions {
    display: flex;
    gap: 7px;
}

.action-button {
    border: none;
    padding:
        8px 11px;
    border-radius: 7px;
    color: white;
    font-size: 13px;
    font-weight: bold;
}

.edit-button {
    background:
        var(--warning);
}

.delete-button {
    background:
        var(--danger);
}

.edit-button:hover,
.delete-button:hover {
    opacity: 0.85;
}
```

```
}
```

```
/* =====  
    EMPTY STATE  
===== */
```

```
.empty-message {  
    text-align: center;  
  
    padding: 40px 20px;  
  
    color:  
        var(--muted);  
  
    display: none;  
}
```

```
.empty-message.show {  
  
    display: block;  
}
```

```
.empty-icon {  
  
    font-size: 45px;  
  
    margin-bottom: 10px;  
}
```

```
/* =====  
    FOOTER / STATS  
===== */
```

```
.task-footer {  
  
    display: flex;  
  
    justify-content:  
        space-between;  
  
    align-items: center;  
  
    gap: 15px;  
  
    flex-wrap: wrap;
```

```

        margin-top: 25px;

        padding-top: 20px;

        border-top:
            1px solid var(--border);
    }

.task-count {

    color:
        var(--muted);

    font-weight: bold;
}

.clear-button {

    background: none;

    border: none;

    color:
        var(--danger);

    font-weight: bold;
}

.clear-button:hover {

    text-decoration:
        underline;
}

/* =====
   EDIT MODAL
   ===== */

.modal {

    position: fixed;

    inset: 0;

    background:

```

```
        rgba(0,0,0,0.55);

display: none;

justify-content: center;

align-items: center;

padding: 20px;

z-index: 1000;
}
```

```
.modal.show {

    display: flex;
}
```

```
.modal-box {

    background: white;

    width: 100%;

    max-width: 500px;

    padding: 30px;

    border-radius: 16px;

    box-shadow:
        0 20px 50px
        rgba(0,0,0,0.2);
}
```

```
.modal-box h2 {

    margin-bottom: 20px;
}
```

```
.modal-box input {

    width: 100%;

    padding: 13px;
```

```
border:
  2px solid var(--border);

border-radius: 9px;

outline: none;

margin-bottom: 20px;
}
```

```
.modal-actions {

  display: flex;

  gap: 10px;

  justify-content: flex-end;
}
```

```
.save-button {

  background:
    var(--success);

  color: white;

  border: none;

  padding:
    10px 20px;

  border-radius: 8px;

  font-weight: bold;
}
```

```
.cancel-button {

  background:
    #64748b;

  color: white;

  border: none;

  padding:
    10px 20px;
```

```

        border-radius: 8px;

        font-weight: bold;
    }

/* =====
   RESPONSIVE DESIGN
   ===== */

@media (max-width: 600px) {

    body {

        padding:
            10px;
    }

    .content {

        padding:
            20px 15px;
    }

    .header {

        padding:
            25px 15px;
    }

    .header h1 {

        font-size:
            28px;
    }

    .add-task {

        flex-direction:
            column;
    }

    .add-button {

```

```
padding:
    13px;
}
```

```
.task-item {

    align-items:
        flex-start;

    flex-wrap:
        wrap;
}
```

```
.task-text {

    min-width:
        60%;
}
```

```
.task-actions {

    width:
        100%;

    justify-content:
        flex-end;
}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<!-- =====
      MAIN APPLICATION
===== -->
```

```
<div class="app">
```

```
<!-- HEADER -->
```

```

<header class="header">

    <h1>
        Smart Task Manager
    </h1>

    <p>
        JavaScript Logic & State Management
    </p>

</header>

<!-- MAIN CONTENT -->

<main class="content">

    <!-- =====
        CREATE
    ===== -->

    <section
        class="add-task"
        aria-label="Add new task">

        <input
            type="text"
            id="taskInput"
            placeholder="Enter a new task..."
            maxlength="150"
            aria-label="New task">

        <button
            class="add-button"
            id="addButton"
            type="button">

            + Add Task

        </button>

    </section>

    <!-- =====
        SEARCH
    ===== -->

    <section class="search-box">

```

```

        <input
            type="search"
            id="searchInput"
            placeholder="🔍 Search tasks..."
            aria-label="Search tasks">

</section>

<!-- =====
      FILTERS
===== -->

<section
    class="filters"
    aria-label="Task filters">

    <button
        class="filter-button active"
        data-filter="all"
        type="button">

        All

    </button>

    <button
        class="filter-button"
        data-filter="active"
        type="button">

        Active

    </button>

    <button
        class="filter-button"
        data-filter="completed"
        type="button">

        Completed

    </button>

</section>

```

```

<!-- =====
      DYNAMIC TASK LIST
===== -->

<section>

    <ul
        class="task-list"
        id="taskList"
        aria-live="polite">

        <!--
            Tasks will be created
            dynamically using JavaScript.
        -->

    </ul>

    <div
        class="empty-message"
        id="emptyMessage">

        <div class="empty-icon">
            
        </div>

        <h3>
            No tasks found
        </h3>

        <p>
            Add a new task to get started.
        </p>

    </div>

</section>

<!-- =====
      TASK FOOTER
===== -->

<footer class="task-footer">

    <span
        class="task-count"
        id="taskCount">

```

```

        0 tasks

    </span>

    <button
        class="clear-button"
        id="clearCompleted"
        type="button">

        Clear Completed

    </button>

</footer>

</main>

</div>

<!-- =====
      EDIT MODAL
===== -->

<div
    class="modal"
    id="editModal"
    role="dialog"
    aria-modal="true"
    aria-labelledby="editTitle">

    <div class="modal-box">

        <h2 id="editTitle">
            Edit Task
        </h2>

        <input
            type="text"
            id="editInput"
            maxlength="150"
            aria-label="Edit task">

        <div class="modal-actions">

```

```

        <button
            class="cancel-button"
            id="cancelEdit"
            type="button">

            Cancel

        </button>

        <button
            class="save-button"
            id="saveEdit"
            type="button">

            Save Changes

        </button>

    </div>

</div>

<!-- =====
      JAVASCRIPT
===== -->

<script>

    /* =====
      STATE
===== */

    let tasks = [];

    let currentFilter = "all";

    let editingTaskId = null;

    /* =====
      DOM ELEMENTS
===== */

    const taskInput =
        document.getElementById(
            "taskInput"

```

```
);

const addButton =
  document.getElementById(
    "addButton"
  );

const taskList =
  document.getElementById(
    "taskList"
  );

const searchInput =
  document.getElementById(
    "searchInput"
  );

const taskCount =
  document.getElementById(
    "taskCount"
  );

const emptyMessage =
  document.getElementById(
    "emptyMessage"
  );

const clearCompleted =
  document.getElementById(
    "clearCompleted"
  );

const editModal =
  document.getElementById(
    "editModal"
  );

const editInput =
  document.getElementById(
    "editInput"
  );
```

```

const saveEdit =
    document.getElementById(
        "saveEdit"
    );

const cancelEdit =
    document.getElementById(
        "cancelEdit"
    );

/* =====
LOCAL STORAGE
===== */

const STORAGE_KEY =
    "smartTaskManagerTasks";

/* =====
LOAD DATA
===== */

function loadTasks() {

    const savedTasks =
        localStorage.getItem(
            STORAGE_KEY
        );

    if (savedTasks) {

        try {

            tasks =
                JSON.parse(
                    savedTasks
                );

        } catch (error) {

            tasks = [];

        }

    }

}

```

```

    renderTasks();
}

/* =====
   SAVE DATA
===== */

function saveTasks() {

    localStorage.setItem(
        STORAGE_KEY,
        JSON.stringify(tasks)
    );
}

/* =====
   CREATE TASK
===== */

function createTask() {

    const text =
        taskInput.value.trim();

    if (text === "") {

        alert(
            "Please enter a task."
        );

        taskInput.focus();

        return;
    }

    const newTask = {

        id:
            Date.now(),

        text:
            text,

        completed:
            false,
    }

```

```

        createdAt:
            new Date().toISOString()
    };

    tasks.push(newTask);

    saveTasks();

    taskInput.value = "";

    renderTasks();

    taskInput.focus();
}

/* =====
   READ / RENDER TASKS
   ===== */

function renderTasks() {

    const searchText =
        searchInput.value
        .trim()
        .toLowerCase();

    let filteredTasks =
        tasks.filter(
            function(task) {

                /* Filter */

                if (
                    currentFilter ===
                    "active" &&
                    task.completed
                ) {

                    return false;
                }
            }
        )
    }

```

```

        if (
            currentFilter ===
            "completed" &&
            !task.completed
        ) {

            return false;
        }

        /* Search */

        if (
            searchText &&
            !task.text
                .toLowerCase()
                .includes(
                    searchText
                )
        ) {

            return false;
        }

        return true;
    }
);

/* Clear old DOM */

taskList.innerHTML = "";

/* Dynamic DOM creation */

filteredTasks.forEach(
    function(task) {

        const li =
            document.createElement(
                "li"
            );

        li.className =
            "task-item";
    }
);

```

```
if (task.completed) {  
    li.classList.add(  
        "completed"  
    );  
}  
  
li.dataset.id =  
    task.id;  
  
/* Checkbox */  
  
const checkbox =  
    document.createElement(  
        "input"  
    );  
  
checkbox.type =  
    "checkbox";  
  
checkbox.className =  
    "task-check";  
  
checkbox.checked =  
    task.completed;  
  
checkbox.setAttribute(  
    "aria-label",  
    "Mark task as completed"  
);  
  
/* Text */  
  
const span =  
    document.createElement(  
        "span"  
    );  
  
span.className =  
    "task-text";
```

```
span.textContent =
    task.text;

/* Actions */

const actions =
    document.createElement(
        "div"
    );

actions.className =
    "task-actions";

/* Edit */

const edit =
    document.createElement(
        "button"
    );

edit.className =
    "action-button edit-button";

edit.dataset.action =
    "edit";

edit.type =
    "button";

edit.textContent =
    "Edit";

/* Delete */

const deleteButton =
    document.createElement(
        "button"
    );

deleteButton.className =
```

```

        "action-button delete-button";

deleteButton.dataset.action =
    "delete";

deleteButton.type =
    "button";

deleteButton.textContent =
    "Delete";

actions.appendChild(
    edit
);

actions.appendChild(
    deleteButton
);

li.appendChild(
    checkbox
);

li.appendChild(
    span
);

li.appendChild(
    actions
);

taskList.appendChild(
    li
);
    }
};

updateTaskCount(
    filteredTasks

```

```

    );

    /* Empty state */

    if (
        filteredTasks.length === 0
    ) {

        emptyMessage.classList.add(
            "show"
        );

    } else {

        emptyMessage.classList.remove(
            "show"
        );

    }

}

/* =====
UPDATE TASK
===== */

function updateTask(
    id,
    newText
) {

    const task =
        tasks.find(
            function(item) {

                return item.id === id;

            }
        );

    if (!task) {
        return;
    }

    task.text =
        newText;

```

```

        saveTasks();

        renderTasks();
    }

    /* =====
    DELETE TASK
    ===== */

    function deleteTask(id) {

        const confirmed =
            confirm(
                "Are you sure you want to delete this task?"
            );

        if (!confirmed) {
            return;
        }

        tasks =
            tasks.filter(
                function(task) {

                    return task.id !== id;

                }
            );

        saveTasks();

        renderTasks();
    }

    /* =====
    TOGGLE COMPLETE
    ===== */

    function toggleTask(id) {

        const task =
            tasks.find(

```

```

        function(item) {
            return item.id === id;
        }
    );

    if (!task) {
        return;
    }

    task.completed =
        !task.completed;

    saveTasks();

    renderTasks();
}

/* =====
   UPDATE COUNT
   ===== */

function updateTaskCount(
    visibleTasks
) {

    const activeTasks =
        tasks.filter(
            function(task) {

                return !task.completed;

            }
        ).length;

    const completedTasks =
        tasks.filter(
            function(task) {

                return task.completed;

            }
        ).length;

```

```

        taskCount.textContent =
            activeTasks +
            " active • " +
            completedTasks +
            " completed • " +
            tasks.length +
            " total";
    }

    /* =====
    EVENT DELEGATION
    ===== */

    taskList.addEventListener(
        "click",
        function(event) {

            const taskItem =
                event.target.closest(
                    ".task-item"
                );

            if (!taskItem) {
                return;
            }

            const taskId =
                Number(
                    taskItem.dataset.id
                );

            const action =
                event.target.dataset.action;

            /* EDIT */

            if (action === "edit") {

                openEditModal(
                    taskId
                );

                return;
            }

```

```

    }

    /* DELETE */

    if (action === "delete") {

        deleteTask(
            taskId
        );

    }

}

);

/* =====
CHECKBOX EVENT
===== */

taskList.addEventListener(
    "change",
    function(event) {

        if (
            event.target.classList
                .contains("task-check")
        ) {

            const taskItem =
                event.target.closest(
                    ".task-item"
                );

            const taskId =
                Number(
                    taskItem.dataset.id
                );

            toggleTask(
                taskId
            );

        }

    }

);

```

```

/* =====
  ADD BUTTON
===== */

addButton.addEventListener(
  "click",
  createTask
);

/* =====
  ENTER KEY
===== */

taskInput.addEventListener(
  "keydown",
  function(event) {

    if (
      event.key ===
      "Enter"
    ) {

      createTask();

    }

  }
);

/* =====
  SEARCH
===== */

searchInput.addEventListener(
  "input",
  function() {

    renderTasks();

  }
);

/* =====
  FILTER
===== */

```

```

document
  .querySelector(".filters")
  .addEventListener(
    "click",
    function(event) {

      if (
        !event.target
        .classList
        .contains(
          "filter-button"
        )
      ) {

        return;
      }

      currentFilter =
        event.target
        .dataset
        .filter;

      document
        .querySelectorAll(
          ".filter-button"
        )
        .forEach(
          function(button) {

            button.classList
              .remove(
                "active"
              );

          }
        );

      event.target.classList
        .add(
          "active"
        );

      renderTasks();

    }
  );

```

```

/* =====
   CLEAR COMPLETED
   ===== */

clearCompleted.addEventListener(
  "click",
  function() {

    tasks =
      tasks.filter(
        function(task) {

          return !task.completed;

        }
      );

    saveTasks();

    renderTasks();

  }
);

/* =====
   OPEN EDIT MODAL
   ===== */

function openEditModal(id) {

  const task =
    tasks.find(
      function(item) {

        return item.id === id;

      }
    );

  if (!task) {
    return;
  }
}

```

```

        editingTaskId =
            id;

        editInput.value =
            task.text;

        editModal.classList.add(
            "show"
        );

        editInput.focus();
    }

    /* =====
    SAVE EDIT
    ===== */

    saveEdit.addEventListener(
        "click",
        function() {

            const newText =
                editInput.value.trim();

            if (
                newText === ""
            ) {

                alert(
                    "Task cannot be empty."
                );

                return;
            }

            updateTask(
                editingTaskId,
                newText
            );

            closeEditModal();
        }
    )

```

```

);

/* =====
   CANCEL EDIT
===== */

cancelEdit.addEventListener(
    "click",
    closeEditModal
);

/* =====
   CLOSE MODAL
===== */

function closeEditModal() {

    editingTaskId =
        null;

    editInput.value =
        "";

    editModal.classList.remove(
        "show"
    );
}

/* =====
   ESCAPE KEY
===== */

document.addEventListener(
    "keydown",
    function(event) {

        if (
            event.key ===
            "Escape"
        ) {

            closeEditModal();

        }

    }
);

```

```
    }  
);  
  
/* =====  
   CLOSE MODAL OUTSIDE  
===== */
```

```
editModal.addEventListener(  
    "click",  
    function(event) {  
        if (  
            event.target ===  
            editModal  
        ) {  
            closeEditModal();  
        }  
    }  
);
```

```
/* =====  
   INITIALIZE APPLICATION  
===== */
```

```
loadTasks();
```

```
</script>
```

```
</body>
```

```
</html>
```